MODULE 6 - Exception Handling in Python

Errors • Exception Handling • Custom Exceptions

Types of Errors in Python

1. Syntax Errors

- Code breaks Python grammar
- Detected before normal execution

Example: missing colon after if

2. Runtime Errors

- Occur while the program runs
- Often raised as exceptions

Example: 10 / 0

3. Logical Errors

- Program runs but gives a wrong result
- Caused by incorrect logic

Example: adding tax twice

Compare the outcomes

```
Syntax error:  if age >= 18 print("Adult")   # missing colon
Runtime error:  result = 10 / 0
Logical error:  total = price - tax   # when tax should be added
```

Basic Exception Handling: try–except

How it works

- Put risky code inside try
- except runs when a matching exception occurs
- The program can respond instead of stopping unexpectedly

Syntax

```
try:
    # code that may fail
except ExceptionType:
    # handle the error
```

Example: Handle division by zero

```
try:
    number = int(input("Enter a number: "))
    print(100 / number)
except ZeroDivisionError:
    print("The number cannot be zero.")
```

else, finally and Multiple Exceptions

try–except–else

The else block runs only when the try block completes without an exception.

Use it for success-only steps.

try–except–finally

The finally block runs whether an exception occurs or not.

Use it for cleanup actions.

Multiple exceptions

Use separate except blocks

Handle specific exceptions before general ones.

Example: else and finally

```
try:
    value = int("25")
except ValueError:
    print("Invalid number")
else:
    print(value)
finally:
    print("Finished")
```

Example: multiple exceptions

```
try:
    number = int(input("Enter a number: "))
    print(1 / number)
except ZeroDivisionError:
    print("You can't divide by zero!")
except ValueError:
    print("Enter only numbers please!")
except Exception:
    print("Something went wrong!")
finally:
    print("Do some cleanup here")
```

Raising Exceptions with raise

Why raise an exception?

- Signal that an input or condition is invalid
- Stop an operation that cannot safely continue
- Make validation rules explicit

Syntax

```
if condition_is_invalid:
    raise ValueError("Helpful error message")
```

Example: Validate an age

```
age = int(input("Enter age: "))
if age < 0:
    raise ValueError("Age cannot be negative.")
print("Age accepted:", age)
```

When Should We Raise an Exception?

Common situations and suitable exceptions

Situation	Exception to raise
Invalid value (e.g., negative age)	**ValueError**
Wrong data type for an operation	**TypeError**
Division by zero	**ZeroDivisionError**
Required file is missing	**FileNotFoundError**
Operation is not permitted	**PermissionError**
Requested item cannot be found	**LookupError**
Application-specific business error	**Custom exception**

Use raise when the current operation cannot proceed and the caller needs to know; ordinary input errors can also be handled with a message and retry.

User-Defined Exceptions

What are custom exceptions?

- Exceptions created for application-specific rules
- Defined as classes that inherit from Exception
- Give errors meaningful names and messages

Basic pattern

```
class InvalidAmountError(Exception):
    pass

raise InvalidAmountError("Amount is invalid")
```

Example: Bank withdrawal validation

```
class InsufficientBalanceError(Exception):
    pass

balance = 5000
amount = 6000
if amount > balance:
    raise InsufficientBalanceError("Insufficient balance for withdrawal")
print("Withdrawal successful")
```

Hands-on: Built-in Exception Handling

Exercise 1: Safe division

Ask the user for two numbers. Handle:

- ValueError for invalid input
- ZeroDivisionError when the second number is zero

Exercise 2: List access

Create a list of five items. Ask for an index and print the item. Handle ValueError and IndexError.

Checklist

☐ Use try and except correctly
☐ Handle only the exceptions you expect
☐ Display a clear message to the user
☐ Test valid input and at least one invalid case